Vertiefung Echtzeitsysteme

Laborübung 3 – RTEMS GPIO-Steuerung mit Threads

Eingereicht von

Mazen Younis

Fachbereich VI – Technische Informatik – Embedded Systems (M. Eng.)
Berliner Hochschule für Technik

Abgabedatum: 24.06.2026

Betreuer
Prof. Dr. Heinz Junkes Berliner Hochschule für Technik

Studiere Zukunft

Laborübung 3 – RTEMS GPIO-Steuerung mit Threads

1 Einleitung und Aufgabenstellung

Ziel dieser Laborübung war die Entwicklung einer RTEMS-Anwendung zur GPIO-Steuerung auf einem Raspberry Pi. Die Anwendung basiert auf der POSIX-API von RTEMS und verwendet mehrere Threads zur parallelen Verarbeitung verschiedener Aufgaben.

Die Aufgabenstellung fordert die Ansteuerung von zwei LEDs über definierte GPIO-Pins des Raspberry Pi. GPIO4 dient als blinkende LED mit konfigurierbarem Blinkintervall. GPIO18 wird bei jeder fallenden Flanke von GPIO4 umgeschaltet. Zusätzlich soll der aktuelle Systemzustand regelmäßig über die serielle Konsole ausgegeben werden.

Die Anwendung wurde in drei voneinander getrennte POSIX-Threads aufgeteilt:

- Thread A erzeugt das zyklische Blinken von GPIO4.
- Thread B verarbeitet Interrupt-Ereignisse und toggelt GPIO18.
- Thread C gibt jede Sekunde den aktuellen Systemzustand auf der Konsole aus.

Zur Synchronisation werden POSIX-Mutexe und POSIX-Semaphoren eingesetzt. Die Implementierung wurde auf einem Raspberry Pi mit Freenove Breakout Board getestet. Die Ausführung und Beobachtung der Konsolenausgabe erfolgte über eine serielle Verbindung mit dem Programm `screen`.

2 Hardwareaufbau und GPIO-Zuordnung

Für die Durchführung der Laborübung wurde ein Raspberry Pi mit einem Freenove Breakout Board verwendet. Die LEDs wurden direkt über die GPIO-Pins des Raspberry Pi angesteuert.

GPIO	Bezeichnung	Richtung	Funktion
GPIO4	LED A	Ausgang	Zyklisches Blinken mit konfigurierbarem Intervall
GPIO18	LED B	Ausgang	Toggle bei jeder fallenden Flanke von GPIO4
GPIO17	Interrupt-Eingang	Eingang	Erkennung der fallenden Flanke über Drahtbrücke von GPIO4

Tab. 1: GPIO-Zuordnung der Anwendung

GPIO4 wurde mit GPIO17 verbunden. Dadurch liegt die von GPIO4 erzeugte fallende Flanke direkt am Interrupt-Eingang GPIO17 an. Sobald GPIO4 von HIGH nach LOW wechselt, löst GPIO17 einen Interrupt aus. Dieser Interrupt signalisiert Thread B, dass GPIO18 umgeschaltet werden soll.

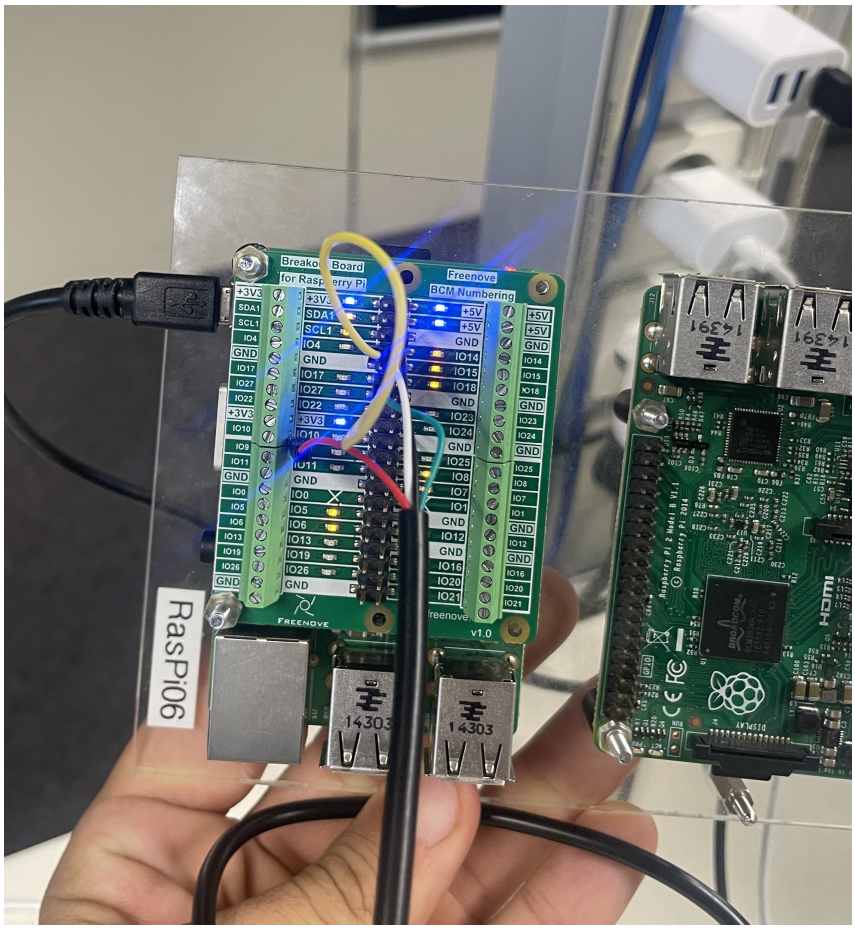


Abb. 1: Hardwareaufbau mit Raspberry Pi, Freenove Breakout Board und Drahtbrücke zwischen GPIO4 und GPIO17

3 Softwarearchitektur

Die Anwendung wurde mit der POSIX-API von RTEMS umgesetzt. Zur Thread-Verwaltung werden POSIX-Threads verwendet. Die Synchronisation erfolgt über einen POSIX-Mutex und ein POSIX-Semaphor.

Beim Start der Anwendung übernimmt der RTEMS-Init-Task die Initialisierung. Zuerst werden die GPIO-Pins konfiguriert. Danach werden die Synchronisationsobjekte erstellt und anschließend die drei POSIX-Threads gestartet.

Thread	Priorität	Aufgabe
Thread A	80	GPIO4 zyklisch ein- und ausschalten
Thread B	70	Auf Interrupt-Ereignis warten, GPIO18 toggeln und Toggle-Zähler erhöhen
Thread C	60	Gemeinsamen Zustand jede Sekunde auf der seriellen Konsole ausgeben

Tab. 2: Übersicht der verwendeten POSIX-Threads

Für die Threads wurde `SCHED_FIFO` verwendet. Bei POSIX bedeutet ein höherer Prioritätswert eine höhere Priorität. Thread A besitzt daher die höchste Priorität, da dieser Thread das zeitliche Verhalten von GPIO4 bestimmt. Thread B verarbeitet die Interrupt-Ereignisse. Thread C besitzt die niedrigste Priorität, da die Konsolenausgabe nicht zeitkritisch ist.

Die Aufteilung in drei Threads macht die Anwendung übersichtlich. Das Blinken, die Interrupt-Verarbeitung und die Ausgabe werden getrennt behandelt, arbeiten aber über gemeinsame Zustandsvariablen zusammen.

4 Synchronisation und gemeinsame Zustände

Mehrere Threads greifen auf gemeinsame Zustandsvariablen zu. Dazu gehören das aktuelle Blinkintervall, der Zustand von GPIO4, der Toggle-Zähler von GPIO18 und die Laufzeit in Sekunden. Diese Werte werden durch den Mutex `mutex_state` geschützt.

Die Kommunikation zwischen Interrupt-Handler und Thread B erfolgt über das POSIX-Semaphor `sem_isr_event`. Der Interrupt-Handler gibt das Semaphor mit `sem_post()` frei. Thread B wartet blockierend mit `sem_wait()` auf dieses Ereignis.

Dadurch wird kein Polling verwendet. Thread B wird nur dann aktiv, wenn tatsächlich eine fallende Flanke an GPIO17 erkannt wurde.

Der Ablauf der Kommunikation ist:

1. Thread A schaltet GPIO4 von HIGH nach LOW.
2. GPIO17 erkennt diese fallende Flanke über die Drahtbrücke.
3. Der Interrupt-Handler `gpio17_isr()` wird ausgeführt.
4. Der Interrupt-Handler gibt das Semaphor `sem_isr_event` frei.
5. Thread B wird geweckt.
6. Thread B toggelt GPIO18 und erhöht den Toggle-Zähler.
7. Thread C liest die gemeinsamen Werte geschützt aus und gibt sie aus.

5 Wichtige Implementierungsdetails

Die Anwendung basiert auf drei POSIX-Threads und einem GPIO-Interrupt. Die eigentliche Funktionalität wird durch die Zusammenarbeit dieser Komponenten umgesetzt.

5.1 Interrupt-Handler

Die Erkennung der fallenden Flanke erfolgt über GPIO17. Sobald GPIO4 von HIGH nach LOW wechselt, wird der Interrupt-Handler aufgerufen.

Der Interrupt-Handler selbst enthält bewusst nur sehr wenig Logik. Er signalisiert lediglich das Auftreten eines Ereignisses über das Semaphor `sem_isr_event`.

```
static rtems_gpio_irq_state gpio17_isr(void *arg)
{
    (void)arg;
    sem_post(&sem_isr_event);
    return IRQ_HANDLED;
}
```

Listing 1: GPIO17 Interrupt-Handler

Dadurch bleibt die Interrupt-Behandlung kurz und blockiert das System nicht unnötig.

5.2 Thread A

Thread A steuert das zyklische Blinken von GPIO4. Vor jeder Blinkphase wird das aktuelle Blinkintervall aus der gemeinsamen Variable `blink_interval_ms` gelesen.

Anschließend wird GPIO4 gesetzt beziehungsweise gelöscht und für die entsprechende Zeit gewartet.

```
rtems_gpio_set(GPIO_LED_A);
sleep_ms(interval_ms);
```

Listing 2: GPIO4 einschalten

Beim Übergang von HIGH nach LOW entsteht die fallende Flanke, die den Interrupt auslöst.

5.3 Thread B

Thread B wartet blockierend auf das Semaphor `sem_isr_event`. Sobald ein Ereignis vorliegt, wird GPIO18 umgeschaltet und der Toggle-Zähler erhöht.

```
sem_wait(&sem_isr_event);
```

Listing 3: Warten auf Interrupt-Ereignis

Dadurch reagiert Thread B ausschließlich auf echte Interrupt-Ereignisse.

5.4 Thread C

Thread C erzeugt jede Sekunde eine Statusausgabe auf der seriellen Konsole. Dabei werden die gemeinsamen Zustandsvariablen unter Mutex-Schutz gelesen.

Die Ausgabe enthält:

- aktuelle Laufzeit
- Zustand von GPIO4
- Anzahl der GPIO18-Umschaltungen
- aktuelles Blinkintervall

Dadurch kann das Verhalten der Anwendung während der Laufzeit beobachtet werden.

6 Laufzeitänderung des Blinkintervalls

Eine zusätzliche Funktion der Anwendung besteht darin, das Blinkintervall während der Laufzeit über die serielle Konsole zu verändern. Dadurch kann das Verhalten von GPIO4 angepasst werden, ohne das Programm neu zu starten.

Der Init-Task liest kontinuierlich Eingaben von der seriellen Konsole ein. Wird eine gültige Zahl zwischen 1 und 1000 eingegeben, wird dieser Wert als neues Blinkintervall übernommen.

```
while (fgets(line, sizeof(line), stdin) != NULL) {

    char *end;
    long val = strtol(line, &end, 10);

    if (end == line) {
        continue;
    }

    if (val < 1 || val > 1000) {
        continue;
    }

    set_blink_interval((uint32_t)val);
}
```

Listing 4: Einlesen eines neuen Blinkintervalls

Die eigentliche Änderung erfolgt in der Funktion `set_blink_interval()`. Der Zugriff auf die gemeinsame Variable `blink_interval_ms` wird dabei durch den Mutex `mutex_state` geschützt.

Dadurch können Thread A, Thread B und Thread C parallel weiterlaufen, während das Blinkintervall sicher aktualisiert wird.

Während der Tests wurden verschiedene Werte eingegeben, beispielsweise 500 ms, 300 ms, 200 ms und 100 ms. Die Änderungen wurden sofort übernommen und waren direkt an der steigenden Anzahl der GPIO18-Umschaltungen erkennbar.

7 GPIO-Initialisierung und Interrupt-Konfiguration

Zu Beginn der Programmausführung wird das GPIO-Subsystem von RTEMS initialisiert. Anschließend werden die benötigten GPIO-Pins konfiguriert.

GPIO4 und GPIO18 werden als digitale Ausgänge verwendet. GPIO17 wird als digitaler Eingang konfiguriert und dient als Interrupt-Eingang für die Erkennung der fallenden Flanke.

```
rtems_gpio_request_pin(
    GPIO_LED_A,
    DIGITAL_OUTPUT,
    false,
    false,
    NULL);
```

```
7
8  rtems_gpio_request_pin(
9      GPIO_LED_B,
10     DIGITAL_OUTPUT,
11     false,
12     false,
13     NULL);
14
15 rtems_gpio_request_pin(
16     GPIO_IRQ_IN,
17     DIGITAL_INPUT,
18     false,
19     false,
20     NULL);
```

Listing 5: Konfiguration der GPIO-Pins

Nach der GPIO-Konfiguration wird der Interrupt für die fallende Flanke von GPIO17 aktiviert.

```
1  rtems_gpio_enable_interrupt(
2      GPIO_IRQ_IN,
3      FALLING_EDGE,
4      UNIQUE_HANDLER,
5      false,
6      gpio17_isr,
7      NULL);
```

Listing 6: Aktivierung des GPIO-Interrupts

Damit wird der Interrupt-Handler automatisch ausgeführt, sobald GPIO17 eine fallende Flanke erkennt.

8 Fehlerbehandlung

Für die Fehlerbehandlung wurde das Makro `CHECK_POSIX` implementiert. Es überprüft die Rückgabewerte von POSIX-Funktionen wie `pthread_create()`, `pthread_mutex_lock()` oder `pthread_mutex_unloc`

Tritt ein Fehler auf, wird eine entsprechende Fehlermeldung auf der Konsole ausgegeben und die Anwendung kontrolliert beendet.

```
1  #define CHECK_POSIX(rc, msg) \
2  do { \
3      int _rc = (rc); \
4      if (_rc != 0) { \
5          printk("[FEHLER] %s: %s\n",
6                 __func__, msg);
7          rtems_fatal_error_occurred(
8              (uint32_t)_rc);
9      } \
10 } while (0)
```

Listing 7: Makro zur Fehlerbehandlung

Zusätzlich werden auch die Rückgabewerte der GPIO-Funktionen überprüft. Dadurch können Fehler bei der Initialisierung oder Konfiguration früh erkannt werden.

Die zentrale Fehlerbehandlung erleichtert die Fehlersuche und erhöht die Robustheit der Anwendung.

9 Build und Ausführung

Die Anwendung wurde für das RTEMS-BSP `arm/raspberrypi2` erstellt.

Der Quellcode wurde in das RTEMS-Beispielprojekt eingebunden und anschließend mit dem Waf-Buildsystem kompiliert.

```
cd ~/development/rtems/examples
./waf build
```

Listing 8: Build der Anwendung

Nach erfolgreicher Übersetzung wurde die ausführbare Datei in ein Raspberry-Pi-Bootimage umgewandelt.

```
arm-rtems7-objcopy -O binary \
gpio_blink.exe \
gpio_blink.img
```

Listing 9: Erzeugung des Kernel-Images

Das erzeugte Image wurde anschließend als `kernel.img` auf die Boot-Partition der SD-Karte kopiert.

Für die Kommunikation mit dem Raspberry Pi wurde die serielle Konsole verwendet.

```
screen /dev/cu.usbserial-10 115200
```

Listing 10: Start der seriellen Konsole

Über diese Verbindung konnten die Programmausgabe beobachtet und neue Blinkintervalle eingegeben werden.

10 Testergebnisse

Zur Validierung der entwickelten RTEMS-Anwendung wurden mehrere Tests auf realer Raspberry-Pi-Hardware durchgeführt. Ziel der Tests war es, die korrekte Funktion der GPIO-Ansteuerung, der Interrupt-Verarbeitung, der Thread-Synchronisation sowie der Laufzeitänderung des Blinkintervalls nachzuweisen.

Die Anwendung wurde auf einem Raspberry Pi mit Freenove Breakout Board ausgeführt. Die Programmausgabe wurde über die serielle Konsole mit dem Werkzeug `screen` beobachtet. Zusätzlich wurde das Verhalten der LEDs direkt auf der Hardware überprüft.

Im Rahmen der Tests wurden insbesondere folgende Punkte untersucht:

- korrektes zyklisches Schalten von GPIO4 durch Thread A,
- zuverlässige Erkennung der fallenden Flanke an GPIO17,
- Aktivierung von Thread B über das POSIX-Semaphor,
- Umschalten von GPIO18 nach jedem Interrupt-Ereignis,
- korrekte Aktualisierung des Toggle-Zählers,
- periodische Statusausgabe durch Thread C,
- sichere Synchronisation gemeinsamer Zustandsvariablen,
- Änderung des Blinkintervalls während der Laufzeit ohne Neustart.

10.1 Hardware-Test

Die Anwendung wurde erfolgreich auf dem Raspberry Pi ausgeführt. Abbildung 2 zeigt den verwendeten Hardwareaufbau mit Raspberry Pi, Freenove Breakout Board sowie der Drahtbrücke zwischen GPIO4 und GPIO17.

Während der Ausführung konnte beobachtet werden, dass GPIO4 entsprechend dem eingestellten Blinkintervall zyklisch geschaltet wird. Die zweite LED an GPIO18 wurde bei jeder erkannten fallenden Flanke von GPIO4 umgeschaltet. Dadurch konnte die korrekte Funktion der Interrupt-Verarbeitung nachgewiesen werden.

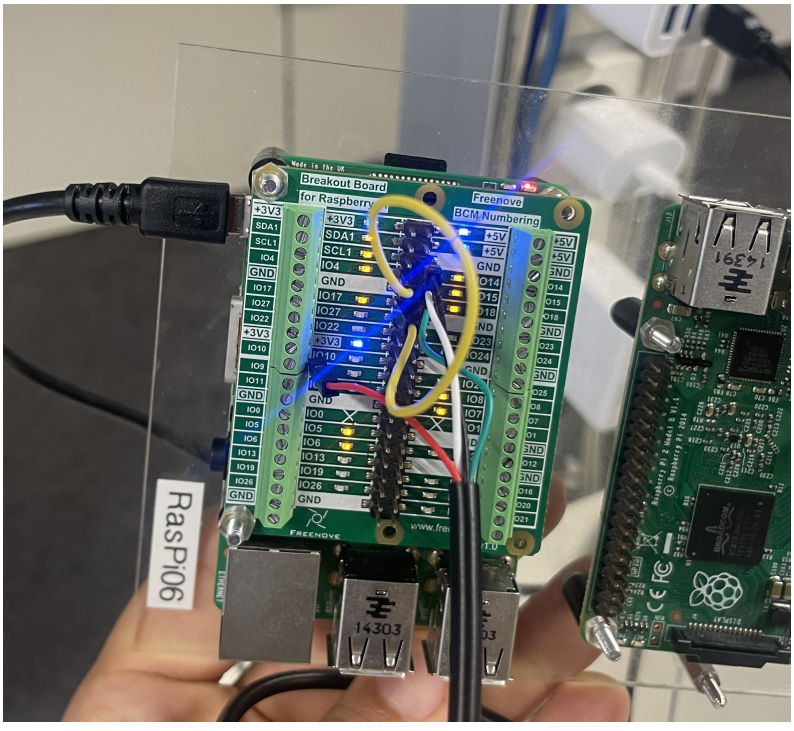


Abb. 2: Testaufbau während der Ausführung auf dem Raspberry Pi

10.2 Überprüfung der Konsolenausgabe

Zur Beobachtung des Systemverhaltens wurde die serielle Konsole verwendet. Thread C erzeugt dabei jede Sekunde eine Statusmeldung mit den wichtigsten Zustandsinformationen der Anwendung.

Die Ausgabe enthält:

- die bisherige Laufzeit,
- den aktuellen Zustand von GPIO4,
- die Anzahl der Umschaltungen von GPIO18,
- das aktuell verwendete Blinkintervall.

Abbildung 3 zeigt einen Ausschnitt der laufenden Anwendung bei einem Blinkintervall von ungefähr 1000 ms.

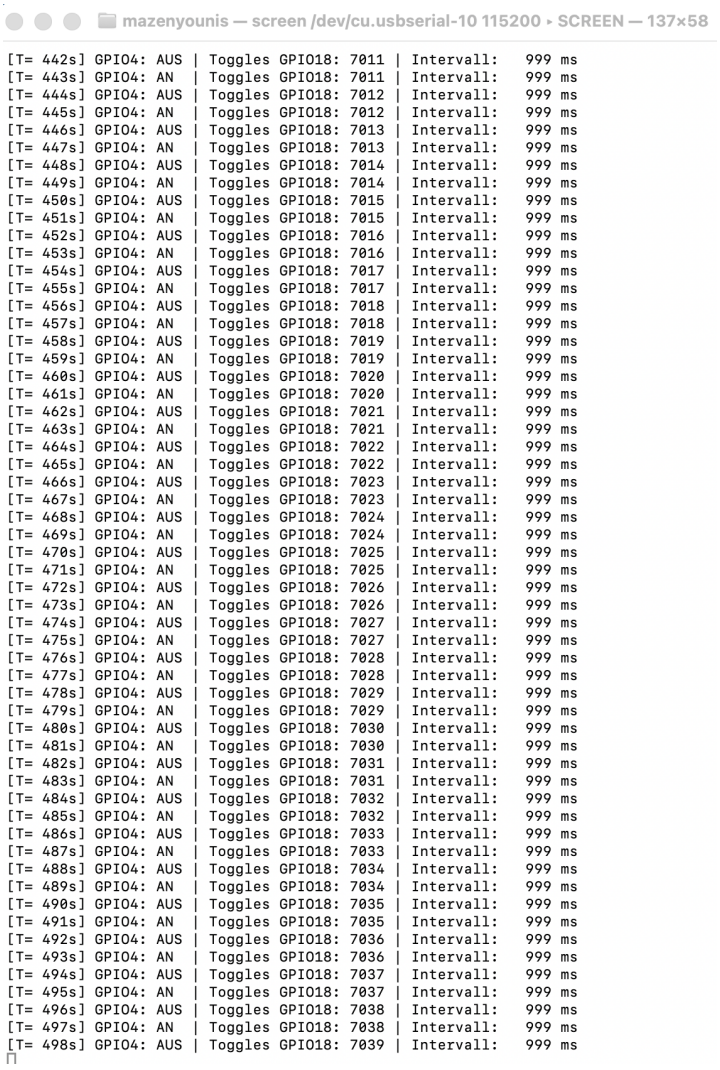


Abb. 3: Serielle Konsolenausgabe bei einem Blinkintervall von etwa 1000 ms

In der Ausgabe ist erkennbar, dass GPIO4 regelmäßig zwischen den Zuständen AN und AUS wechselt. Gleichzeitig steigt der Toggle-Zähler kontinuierlich an. Dies bestätigt, dass die Zusammenarbeit zwischen Thread A, Interrupt-Handler, Thread B und Thread C korrekt funktioniert.

10.3 Test der Laufzeitänderung

Eine zentrale Anforderung der Laborübung bestand darin, das Blinkintervall während der Programmausführung ändern zu können. Hierzu wurden nacheinander die Werte 500 ms, 300 ms, 200 ms und 100 ms über die serielle Konsole eingegeben.

Die Anwendung überprüft die Eingabewerte und übernimmt gültige Werte unmittelbar in die globale Variable für das Blinkintervall. Ein Neustart der Anwendung ist hierfür nicht erforderlich.

Abbildung 4 zeigt die erfolgreiche Änderung des Blinkintervalls während der Laufzeit.

```
[INFO] Blinkintervall geaendert: 200 ms
[T= 514s] GPIO4: AN  | Toggles GPIO18: 7048 | Intervall:   200 ms
[T= 515s] GPIO4: AUS | Toggles GPIO18: 7051 | Intervall:   200 ms
[T= 516s] GPIO4: AN  | Toggles GPIO18: 7053 | Intervall:   200 ms
[T= 517s] GPIO4: AUS | Toggles GPIO18: 7056 | Intervall:   200 ms
[T= 518s] GPIO4: AN  | Toggles GPIO18: 7058 | Intervall:   200 ms
[T= 519s] GPIO4: AUS | Toggles GPIO18: 7061 | Intervall:   200 ms
[T= 520s] GPIO4: AN  | Toggles GPIO18: 7063 | Intervall:   200 ms
[T= 521s] GPIO4: AUS | Toggles GPIO18: 7066 | Intervall:   200 ms
[T= 522s] GPIO4: AN  | Toggles GPIO18: 7068 | Intervall:   200 ms
[T= 523s] GPIO4: AUS | Toggles GPIO18: 7071 | Intervall:   200 ms
[T= 524s] GPIO4: AN  | Toggles GPIO18: 7073 | Intervall:   200 ms
[T= 525s] GPIO4: AUS | Toggles GPIO18: 7076 | Intervall:   200 ms
[T= 526s] GPIO4: AN  | Toggles GPIO18: 7078 | Intervall:   200 ms
[T= 527s] GPIO4: AUS | Toggles GPIO18: 7081 | Intervall:   200 ms
[T= 528s] GPIO4: AN  | Toggles GPIO18: 7083 | Intervall:   200 ms
[T= 529s] GPIO4: AUS | Toggles GPIO18: 7086 | Intervall:   200 ms
100[T= 530s] GPIO4: AN  | Toggles GPIO18: 7088 | Intervall:   200 ms

[INFO] Blinkintervall geaendert: 100 ms
[T= 531s] GPIO4: AN  | Toggles GPIO18: 7093 | Intervall:   100 ms
[T= 532s] GPIO4: AUS | Toggles GPIO18: 7098 | Intervall:   100 ms
[T= 533s] GPIO4: AUS | Toggles GPIO18: 7103 | Intervall:   100 ms
[T= 534s] GPIO4: AUS | Toggles GPIO18: 7108 | Intervall:   100 ms
[T= 535s] GPIO4: AUS | Toggles GPIO18: 7113 | Intervall:   100 ms
```

Abb. 4: Änderung des Blinkintervalls während der Laufzeit

Nach der Eingabe neuer Werte wird das aktualisierte Intervall unmittelbar in der Statusausgabe angezeigt. Gleichzeitig verändert sich die Geschwindigkeit der Umschaltungen von GPIO4 und damit auch die Anzahl der von Thread B verarbeiteten Interrupt-Ereignisse.

Die Ergebnisse zeigen, dass die Synchronisation der gemeinsamen Zustandsvariablen korrekt funktioniert und Änderungen des Blinkintervalls ohne Unterbrechung des Programmbetriebs möglich sind.

10.4 Zusammenfassung der Testergebnisse

Alle vorgesehenen Funktionstests wurden erfolgreich durchgeführt. Die Anwendung zeigte auf realer Hardware das erwartete Verhalten.

Insbesondere konnte nachgewiesen werden, dass:

- GPIO4 mit dem vorgegebenen Intervall geschaltet wird,
- fallende Flanken an GPIO17 zuverlässig erkannt werden,
- Interrupt-Ereignisse korrekt an Thread B weitergegeben werden,
- GPIO18 bei jedem Ereignis umgeschaltet wird,
- der Toggle-Zähler konsistent aktualisiert wird,
- Thread C den aktuellen Systemzustand periodisch ausgibt,
- Änderungen des Blinkintervalls während der Laufzeit wirksam werden.

Damit wurden sämtliche funktionalen Anforderungen der Laborübung erfolgreich erfüllt.

11 Git-Repository

Für die Abgabe wurde zusätzlich ein Git-Repository erstellt. Das Repository enthält den Quellcode der Anwendung, die Dokumentation, die verwendeten Bilder sowie die Testergebnisse.

Das Repository ist unter folgendem Link verfügbar:

https://github.com/MazenYounis/VE_Labor3

12 Fazit

Im Rahmen dieser Laborübung wurde eine RTEMS-Anwendung zur GPIO-Steuerung mit mehreren POSIX-Threads entwickelt und erfolgreich auf einem Raspberry Pi mit Freenove Breakout Board umgesetzt.

Die Anwendung verwendet drei Threads mit unterschiedlichen Aufgaben. Thread A erzeugt das zyklische Blinken von GPIO4. Über die Drahtbrücke zwischen GPIO4 und GPIO17 werden fallende Flanken erkannt und als Interrupt-Ereignisse verarbeitet. Der Interrupt-Handler signalisiert diese Ereignisse über ein POSIX-Semaphor an Thread B, welcher GPIO18 umschaltet und den Toggle-Zähler aktualisiert. Thread C überwacht den aktuellen Systemzustand und gibt die wichtigsten Informationen periodisch auf der seriellen Konsole aus.

Zur Synchronisation gemeinsamer Zustandsvariablen wurden POSIX-Mutexe eingesetzt. Dadurch konnten konkurrierende Zugriffe sicher behandelt werden. Die Kommunikation zwischen Interrupt-Kontext und Anwendungsebene wurde mithilfe eines POSIX-Semaphores realisiert. Das Ergebnis ist eine klare Trennung zwischen Hardware-Ereignissen und deren Verarbeitung in den Threads.

Die durchgeführten Tests auf realer Hardware bestätigten die korrekte Funktion aller Komponenten. Insbesondere konnte nachgewiesen werden, dass Interrupt-Ereignisse zuverlässig erkannt werden, die Thread-Kommunikation fehlerfrei funktioniert und Änderungen des Blinkintervalls während der Laufzeit unmittelbar wirksam werden.

Damit wurden sämtliche Anforderungen der Laborübung erfolgreich erfüllt. Gleichzeitig konnte ein praktischer Einblick in die Entwicklung von Echtzeitanwendungen mit RTEMS, POSIX-Threads, Semaphoren und Interrupt-Verarbeitung gewonnen werden.